অধ্যায় ২৬

Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture

PostgreSQL, Prisma এবং TravelData Pro

শেখার লক্ষ্য

এই অধ্যায় শেষে relational schema, keys, constraints, normalization, transactions, isolation, ORM, migrations, indexes, query plans, pagination, connection pools, caching এবং scalable database design ব্যাখ্যা ও প্রয়োগ করতে পারবেন। Chapter 25-এর in-memory backend-কে স্থায়ী relational persistence-এ উন্নীত করার নকশা তৈরি করব।

২৬.১ Database কেন গুরুত্বপূর্ণ?

Chapter 25-এ server restart হলে in-memory bookings হারিয়ে যায়। Production booking system-এ account, booking, inventory ও payment-এর মধ্যে নির্ভরযোগ্য সম্পর্ক দরকার। PostgreSQL-এর মতো relational database ACID transaction, foreign keys, uniqueness এবং concurrent update-এর জন্য উপযোগী। তবে database ব্যবহার করলেই correctness নিশ্চিত হয় না: application business rules, authorization, idempotency এবং failure recovery-ও জরুরি।

উদাহরণ: ১টি আসন বাকি, একই সময়ে দুইজন বুকিং করলেন। আগে availability পড়ে পরে update করলে দুজনই সফল হতে পারেন। Database transaction এবং conditional update দিয়ে overselling আটকাতে হবে।

২৬.২ Relational Modeling ও Normalization

মূল entity: `User`, `TravelPackage`, `Booking`। একটি User-এর বহু Booking থাকতে পারে, একটি Package-এর বহু Booking থাকতে পারে। প্রতিটি Booking একটি User এবং একটি Package-এর সঙ্গে সম্পর্কিত।

Entity	গুরুত্বপূর্ণ ক্ষেত্র	Constraint
User	id, email, passwordHash	email UNIQUE, NOT NULL
TravelPackage	id, title, priceMinor, availableSeats	price ও seats >= 0
Booking	id, userId, packageId, seats, status, totalMinor	Foreign keys, seats > 0

Normalization: একই package title বা customer email প্রতিটি Booking row-তে কপি করলে update anomaly তৈরি হয়। আলাদা entity এবং foreign key ব্যবহার করলে data duplication কমে। তবে reporting-এর জন্য deliberate denormalization প্রয়োজন হতে পারে—সেটি পরিমাপ ও documentation করে করতে হবে। Booking-

এর `totalMinor`-এ purchase-time price snapshot রাখা যুক্তিযুক্ত, কারণ পরে package price বদলালেও পুরোনো booking total বদলানো উচিত নয়।

২৬.৩ PostgreSQL Schema এবং Constraints

```sql
CREATE TABLE app_users (
  id UUID PRIMARY KEY,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE travel_packages (
  id UUID PRIMARY KEY,
  title TEXT NOT NULL,
  price_minor BIGINT NOT NULL CHECK (price_minor >= 0),
  available_seats INTEGER NOT NULL CHECK (available_seats >= 0)
);
CREATE TABLE bookings (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES app_users(id),
  package_id UUID NOT NULL REFERENCES travel_packages(id),
  seats INTEGER NOT NULL CHECK (seats > 0),
  total_minor BIGINT NOT NULL CHECK (total_minor >= 0),
  status TEXT NOT NULL CHECK (status IN ('CONFIRMED', 'CANCELLED')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX bookings_user_created_idx ON bookings (user_id, created_at DESC);
```

Money `float` দিয়ে গণনা না করে minor units (যেমন পয়সা) বা নির্দিষ্ট decimal type ব্যবহার করা ভালো। `BIGINT` JavaScript-এ `BigInt` হিসেবে আসতে পারে; JSON serialization-এর সময় explicit conversion strategy লাগবে। UUID collision resistance যথেষ্ট হলেও authorization-এর বিকল্প নয়।

২৬.৪ SQL Query ও Injection Prevention

```sql
SELECT b.id, b.status, b.total_minor, p.title
FROM bookings b
JOIN travel_packages p ON p.id = b.package_id
WHERE b.user_id = $1
ORDER BY b.created_at DESC, b.id DESC
LIMIT $2;
```

`$1`, `$2` parameter binding-এর উদাহরণ; কখনো user input string concatenate করে SQL তৈরি করবেন না। Dynamic sort column-এর ক্ষেত্রে parameter placeholder দিয়ে identifier bind করা যায় না; আগে নির্দিষ্ট allowlist ব্যবহার করুন।

২৬.৫ ORM: Prisma কীভাবে কাজ করে?

Prisma schema-তে model, relation এবং index বর্ণনা করা হয়; generated client থেকে type-aware query করা যায়। ORM সুবিধাজনক হলেও generated SQL ও database constraints বোঝা জরুরি। এই অধ্যায়ের `source-code/chapter-26/prisma/schema.prisma`-তে বাস্তব model আছে।

```typescript
const bookings = await prisma.booking.findMany({
  where: { userId: currentUserId },
  orderBy: [{ createdAt: 'desc' }, { id: 'desc' }],
  take: 20,
  select: { id: true, status: true, totalMinor: true,
    travelPackage: { select: { title: true } } }
});
```

এই query server-side authorized user ID দিয়ে filter করতে হবে; browser পাঠানো arbitrary user ID বিশ্বাস করা যাবে না। Schema-এর enum/relations এবং generated client-এর নাম সংস্করণভেদে যাচাই করুন।

২৬.৬ Migrations এবং Data Lifecycle

Schema পরিবর্তনের জন্য version-controlled migration দরকার। Development-এ `prisma migrate dev`, deployment pipeline-এ `prisma migrate deploy` ব্যবহৃত হয়। Production-এ destructive migration দেওয়ার আগে backup, rollback/roll-forward plan এবং data compatibility পরীক্ষা করতে হবে।

Expand–Migrate–Contract: নতুন nullable column যোগ করুন → app উভয় version support করুক → data backfill করুন → constraint দিন → পুরোনো column সরান। Zero-downtime deployment-এ এই কৌশল কার্যকর।

২৬.৭ Transaction, Atomicity ও Concurrency

নিচের pattern-এ conditional inventory decrement ও booking insert একই transaction-এ করা হয়:

```typescript
await prisma.$transaction(async (tx) => {
  const updated = await tx.travelPackage.updateMany({
    where: { id: packageId, availableSeats: { gte: seats } },
    data: { availableSeats: { decrement: seats } }
  });
  if (updated.count !== 1) throw new Error('SOLD_OUT');
  const pkg = await tx.travelPackage.findUniqueOrThrow({ where: { id: packageId } });
  await tx.booking.create({ data: {
    userId, packageId, seats,
    totalMinor: pkg.priceMinor * BigInt(seats),
    status: 'CONFIRMED'
  }});
});
```

গুরুত্বপূর্ণ: এটি illustrative code; transaction-এর বাইরে price snapshot নেওয়া, retry policy, concurrent price changes, idempotency key, cancellation refund এবং authorization production system-এ আলাদাভাবে design করতে হবে। Conditional update database-এ atomically execute হয়। Unique idempotency key দিয়ে duplicate network retry-ও আটকানো যায়।

২৬.৮ Isolation Levels এবং Deadlocks

Read Committed, Repeatable Read ও Serializable-এর পার্থক্য transaction anomaly-তে প্রকাশ পায়। Serializable শক্তিশালী guarantee দেয়, কিন্তু serialization failure হলে transaction retry দরকার। Deadlock-এর ঝুঁকি কমাতে একই resource lock order অনুসরণ করুন, transaction ছোট রাখুন এবং external HTTP call transaction-এর মধ্যে করবেন না।

২৬.৯ Index, EXPLAIN এবং Query Optimization

Index lookup দ্রুত করতে পারে, কিন্তু write/storage cost বাড়ায়। Query pattern অনুযায়ী composite index নির্বাচন করুন। PostgreSQL `EXPLAIN (ANALYZE, BUFFERS)` বাস্তব query চালিয়ে plan ও cost দেখায়—production-এ সাবধানতা প্রয়োজন। `SELECT *` এড়িয়ে প্রয়োজনীয় field নিন, N+1 query কমান এবং filter/sort-এর জন্য index design করুন।

২৬.১০ Pagination, Pooling ও Caching

`OFFSET` pagination বড় table-এ ধীর হতে পারে এবং concurrent insert-এ page drift হয়। Stable ordering ও cursor/keyset pagination বিবেচনা করুন। Connection pool সীমিত রাখুন; প্রতিটি request-এ নতুন database connection তৈরি করবেন না। Cache ব্যবহার করলে TTL, invalidation, stale-data tolerance এবং privacy নির্ধারণ করুন। Booking confirmation-এর মতো critical decision cache থেকে করবেন না।

২৬.১১ Backup, Security ও Observability

Database credential environment secret manager-এ রাখুন। Least-privilege DB user, TLS, encryption at rest, access audit, backups ও restore drills রাখুন। Query duration, error rate, lock wait, connection saturation এবং slow query পর্যবেক্ষণ করুন। শুধু backup নেওয়া যথেষ্ট নয়—restore করে পরীক্ষা করা জরুরি।

২৬.১২ Practical Project — TravelData Pro

Architecture: HTTP API → Booking Service → Repository → PostgreSQL/Prisma। `source-code/chapter-26/`-তে Prisma schema, SQL DDL, transaction-oriented example এবং **dependency-free in-memory simulation** দেওয়া আছে। Simulation-এ stock reservation, cancellation, duplicate request protection, reporting এবং cursor pagination পরীক্ষা করা যায়। এটি real PostgreSQL-এর সমতুল্য concurrency guarantee দেয় না।

Run

```
TERMINAL
cd source-code/chapter-26
node --test
node demo.mjs
```

Real Database Setup (ঐচ্ছিক)

```
TERMINAL
npm install
# PostgreSQL instance চালু করুন এবং DATABASE_URL সেট করুন
npx prisma migrate dev --name init
npx prisma generate
```

Migration তৈরি ও চালাতে PostgreSQL service প্রয়োজন। এই বইয়ের bundle-এ live database বা credentials নেই। `prisma/schema.prisma` ব্যবহার করার আগে নিজের environment-এর Prisma version-এর configuration নিয়ম যাচাই করুন।

২৬.১৩ Exercises

১. User–Booking relation-এর cardinality ব্যাখ্যা করুন।
২. `email`-এ UNIQUE constraint কেন দরকার?
৩. `price` float ব্যবহার করলে কী সমস্যা হতে পারে?
৪. N+1 query কীভাবে শনাক্ত করবেন?
৫. `availableSeats` 0-এর নিচে নামা কীভাবে রোধ করবেন?
৬. Offset এবং cursor pagination-এর tradeoff লিখুন।
৭. Booking cancellation-এ transaction কেন দরকার?
৮. SQL injection রোধের দুটি নিয়ম লিখুন।
৯. Index থাকলেও query slow হওয়ার তিনটি কারণ লিখুন।
১০. Expand–Migrate–Contract দিয়ে একটি field rename-এর plan লিখুন।

২৬.১৪ Sample Solutions

১. One User → Many Bookings; Each Booking → One User।
২. Duplicate account identity database স্তরে রোধ করে; app validation-এর race condition-ও আটকায়।
৩. Binary floating-point rounding error; minor units বা fixed decimal ব্যবহার করুন।
৪. Query log/trace-এ একটি parent query-এর পরে প্রতি row-এর জন্য child query দেখলে N+1 সন্দেহ। Eager loading/batched query ব্যবহার করুন।
৫. `CHECK (available_seats >= 0)` এবং conditional atomic update; দুটোই দরকার।
৬. Offset সহজ কিন্তু বড় offset ধীর; cursor দ্রুত ও stable ordering-এ ভালো, তবে implementation জটিল।
৭. Status update ও stock release একসঙ্গে commit/rollback হওয়া দরকার।

৮. Parameterized queries; dynamic identifiers-এর allowlist।
৯. Low selectivity, wrong index/order, stale statistics, expensive joins বা large result set।
১০. New column যোগ → dual read/write → backfill → switch read → constraint → old column drop।

২৬.১৫ MCQ — Self Assessment

১. ACID-এর A? (A) Atomicity (B) Availability (C) Authorization (D) Aggregation
২. Foreign key কী করে? (A) UI render (B) Referential integrity (C) Encryption (D) Caching
৩. SQL injection কমায়? (A) String concatenation (B) Parameter binding (C) eval (D) localStorage
৪. Duplicate email আটকায়? (A) INDEX only (B) UNIQUE (C) ORDER BY (D) LIMIT
৫. N+1-এর উপযুক্ত সমাধান? (A) More loops (B) Batch/eager load (C) Remove constraints (D) No index
৬. Migration deploy কোথায়? (A) Production pipeline (B) CSS (C) Browser only (D) README only
৭. Cursor pagination-এ প্রয়োজন? (A) Unstable order (B) Stable ordering (C) Random order (D) No index
৮. Critical inventory check কোথায়? (A) Browser cache (B) Database transaction (C) CSS (D) Client clock
৯. EXPLAIN ANALYZE কী করে? (A) Query execute ও profile (B) Password hash (C) UI test (D) Git merge
১০. Backup-এর নির্ভরযোগ্যতা কীভাবে জানবেন? (A) File size (B) Restore test (C) Filename (D) Commit count

Answer key: ১-A (Atomicity), ২-B (referential integrity), ৩-B (parameterization), ৪-B (unique constraint), ৫-B (batch loading), ৬-A (deployment), ৭-B (stable order), ৮-B (transaction), ৯-A (execution plan), ১০-B (restore drill)।

২৬.১৬ Chapter Summary ও পরবর্তী অধ্যায়

Database design-এ correctness প্রথম, performance দ্বিতীয়, scaling তৃতীয়—তবে তিনটি একসঙ্গে পরিকল্পনা করতে হয়। Chapter 27-এ booking updates real-time clients-এ পৌঁছাতে WebSockets, Socket.IO এবং event-driven messaging শিখব।